A Sheaf-Theoretic Block-Level Transcoder for Hybrid Transformer–Linear Models

Sparse local codes, learned restriction maps, and cohomological diagnostics

Submission-ready LaTeX scaffold and extended outline

Working title options

1. **A Sheaf-Theoretic Block-Level Transcoder for Hybrid Transformer–Linear Models**
2. **Block-Level Geometric Representation Learning for Hybrid 3:1 Architectures**
3. **From Topological Sidecars to Cohomological Diagnostics in Hybrid Language Models**

Abstract

Hybrid transformer–linear architectures generate representations across distinct computational regimes, but standard topological analysis remains largely post hoc: it summarizes activation geometry after the fact rather than modeling how consistency and obstruction arise across the computation itself. We introduce a sheaf-theoretic block-level transcoder for hybrid models in which hook-window activations are treated as local sections over a cover of the computation graph, and learned restriction maps define local transport across overlapping computational regions. This formulation yields exact cohomological observables—including global section dimension, obstruction classes, sheaf Laplacian spectra, and stalk-level inconsistency—that replace heuristic coactivation and persistence sidecars with architecture-aware local-to-global diagnostics. We instantiate the framework on block-structured hybrid 3:1 architectures, where tract layers compress context through recurrent or linear state updates and bridge layers periodically restore exact retrieval. The resulting geometry exposes bridge necessity, cross-block concept evolution, and intervention-induced obstruction in a way that flat residual-space transcoders do not. The paper contributes (i) a formal sheaf model of hybrid computation, (ii) a trainable block-level geometric transcoder with consistency losses, and (iii) an experimental program for mechanistic interpretability on hybrid language models.

Paper identity

This is a **methods paper**, not a manifesto. The central object is a **block-level geometric transcoder** grounded in a **sheaf-theoretic formalization** of hybrid computation. The imple-

mentation vehicle can be BLT or a BLT-derived system, but the intellectual center of gravity should be:

1. a formal object,
2. a trainable method,
3. a reproducible instrumentation pipeline,
4. and an empirical interpretability program.

1. Introduction

Purpose of the section

State the problem, narrow the contribution, and make it clear that the paper proposes a concrete representation-learning and interpretability method for hybrid transformer–linear architectures.

Core narrative

Classical topological data analysis is useful when the object of study is a fixed dataset or point cloud. It becomes insufficient when the "data" is an evolving hidden-state computation whose geometry changes across heterogeneous computational regimes. Hybrid transformer–linear architectures sharpen this problem: one part of the model compresses context into bounded recurrent or state-space updates, while another part periodically restores exact token-to-token interaction. A post hoc persistence summary of activations does not describe how these regimes coordinate, where they fail to coordinate, or how such failures propagate through the model.

The central claim of this paper is that the computation itself should be modeled as a sheaf. Hook-window activations become local sections, overlaps of computation windows induce restriction maps, and failures of global consistency become cohomological obstructions. On top of this formalism, we introduce a block-level geometric transcoder that learns sparse local codes and restriction maps over hybrid blocks, making consistency, obstruction, and bridge dependence first-class observables rather than after-the-fact heuristics.

Three-sentence problem statement

> Hybrid transformer–linear models induce block-structured computation with multiple local regimes, but existing topological summaries of activations remain largely post hoc. We model block-local hook activations as sections of a sheaf over a cover of the

computation graph, with learned restriction maps capturing transport across overlapping computational regions. This yields a trainable block-level transcoder whose sheaf Laplacian and cohomology provide exact diagnostics for global consistency, irreducible obstruction, and bridge-mediated retrieval.

Contribution bullets

1. We formalize a hybrid forward pass as a cellular sheaf over block-local hook windows.
2. We introduce a block-level geometric transcoder with sparse local codes and learned restriction maps.
3. We derive exact local-to-global diagnostics from the cochain complex and sheaf Laplacian.
4. We design experiments for bridge necessity, compression-forgetting, cross-block concept evolution, and intervention-induced obstruction.

Reviewer-facing note: Make the introduction feel empirical and method-driven. Save the broader TDS/GDS framing for the discussion section or the final paragraph of the introduction.

2. Background and problem setting

2.1 Hybrid transformer–linear architectures

Define the target class of models. A canonical case is a repeating 3:1 block in which three tract layers perform recurrent, state-space, or linearized transport updates and a fourth bridge layer performs full attention or another exact retrieval operation. The tract layers form a compressed transport regime; the bridge layer acts as a global refresh or exact retrieval regime.

Figure 1 placeholder: Hybrid 3:1 block cover
Depict three tract hooks and one bridge hook, with overlaps induced by shared residual stream and block-local computation windows.

Figure 1: Hybrid 3:1 block structure and the corresponding cover by hook windows.

2.2 Limits of post hoc topological summaries

Explain why standard TDA is insufficient here:

1. it treats latent geometry as static rather than computationally generated,

2. it does not encode architectural overlap or transport,
3. it summarizes shape after the fact instead of modeling consistency during computation.

2.3 Why sheaves are the correct local-to-global object

A sheaf is the natural formal object when one has:

1. local data on overlapping regions,
2. restriction maps across inclusions or overlaps,
3. a gluing law for global consistency,
4. and algebraic obstructions when gluing fails.

That is exactly the structure present in a hooked, block-localized hybrid computation.

Reviewer-facing note: This section should read like careful problem setup, not like a survey article. Keep it short and technical.

3. Formal framework

3.1 Base space and cover

Definition 1 (Hook-window cover)**.** *Let X denote the block-structured computation graph of a hybrid model. Define a cover $\mathcal{U} = \{U_i\}_{i=1}^m$ where each open set*

$$U_i = (\textit{layer range}, \textit{hook name}, \textit{block index})$$

corresponds to a hook-window over which an activation tensor is captured.

Each overlap $U_i \cap U_j$ represents a shared computational region: shared tokens, shared residual subspace, or a region of the computation graph on which both hook-windows impose constraints.

3.2 Sections and restriction maps

Definition 2 (Activation sheaf)**.** *A sheaf $\mathcal{F}$ over $\mathcal{U}$ assigns to each U_i a vector space $\mathcal{F}(U_i)$ of admissible activations, typically a subspace of $\mathbb{R}^{T\times d}$ or a sparse-coded representation thereof. For each inclusion or overlap map, $\mathcal{F}$ provides a restriction map*

$$\rho_{U_i,U_j} : \mathcal{F}(U_i) \to \mathcal{F}(U_j)$$

whenever the computational structure defines such a restriction.

For a particular prompt, the model induces a local section

$$s_i \in \mathcal{F}(U_i).$$

3.3 Cochain complex

The hooked activations induce a cochain complex

$$C^0(\mathcal{U};\mathcal{F}) \xrightarrow{\delta^0} C^1(\mathcal{U};\mathcal{F}) \xrightarrow{\delta^1} C^2(\mathcal{U};\mathcal{F}).$$

Interpretation:

- C^0: local sections, one per hook-window;
- C^1: pairwise disagreements over overlaps;
- C^2: loop-level inconsistency or holonomy over triple overlaps.

For a 0-cochain $s = \{s_i\}$, define the first coboundary by

$$(\delta^0 s)_{ij} = \rho_{U_i,U_i\cap U_j}(s_i) - \rho_{U_j,U_i\cap U_j}(s_j).$$

3.4 Cohomology and sheaf Laplacian

Definition 3 (Global sections and obstructions)**.** *The sheaf cohomology groups of interest are*

$$H^0 = \ker(\delta^0), \qquad H^1 = \ker(\delta^1)/\operatorname{im}(\delta^0).$$

Interpretation:

- $\dim H^0$: number of independent globally consistent section patterns;
- $\dim H^1$: irreducible obstruction classes that cannot be resolved by local correction.

Define the degree-0 sheaf Laplacian by

$$L_0 = (\delta^0)^\top \delta^0.$$

Proposition 1. $\ker(L_0) = H^0$.

The smallest nonzero eigenvalue of L_0 acts as a spectral robustness measure for global consistency.

3.5 Stalks and local inconsistency

At a point $x = (\ell, t)$ corresponding to a layer-token position, define the stalk $\mathcal{F}_x$ as the colimit of local section data over all hook-windows containing x. A practical stalk defect metric is

$$\mathrm{defect}(x) = \frac{1}{\binom{|S_x|}{2}} \sum_{(s_i,s_j)\in S_x^2} \|\rho(s_i)|_x - \rho(s_j)|_x\| \,,$$

where S_x denotes the family of local sections participating at x.

Reviewer-facing note: This section is where rigor earns trust. Keep the notation stable and avoid changing the model definition halfway through the paper.

4. Method: Block-Level Geometric Transcoder

4.1 Overview

The method maps hooked activations to sparse block-local codes, learns restriction maps across overlapping hook-windows, and optimizes both reconstruction and sheaf-consistency objectives.

Figure 2 placeholder: Method diagram
Activation capture → sparse local code encoder → learned restriction maps → coboundary / Laplacian losses → diagnostic readouts.

Figure 2: Block-level geometric transcoder pipeline.

4.2 Sparse local codes

For each hook-window U_i, let $a_i \in \mathbb{R}^{T\times d}$ be the captured activation. The encoder maps it to a sparse code

$$z_i = E_\theta(a_i),$$

with either tokenwise or blockwise sparsity constraints. The key conceptual choice is that z_i is not treated as a generic latent vector; it is a local chart or stalk-level representation.

4.3 Learned restriction maps

For overlapping windows (U_i, U_j), define a learned linear or low-rank transport operator

$$\rho_{ij}(z_i) = R_{ij} z_i,$$

with R_{ij} constrained according to architectural prior:

- bridge-to-tract maps may be low-rank or structured,
- tract-to-tract maps may be approximately norm-preserving,
- identity maps serve as a baseline, not the headline method.

4.4 Objective

A natural objective is

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda_\delta \|\delta^0 z\|_2^2 + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}} + \lambda_{\text{reg}} \mathcal{R}(R),$$

where:

$$\mathcal{L}_{\text{recon}} = \sum_i \|D_\phi(z_i) - a_i\|_2^2,$$
$$\|\delta^0 z\|_2^2 = \sum_{(i,j)} \|R_{i\to ij} z_i - R_{j\to ij} z_j\|_2^2 .$$

Possible regularizers:

- sparsity penalties on z_i,
- orthogonality or low-rank constraints on R_{ij},
- spectral penalties encouraging a stable but nontrivial Laplacian gap.

4.5 Derived observables

The trained system reports:

1. $\dim H^0$,
2. $\dim H^1$,

3. spectral gap of L_0,
4. stalk defect maps,
5. bridge dependence / bridge necessity,
6. obstruction energy.

A practical obstruction-energy decomposition is

$$z = z_{\text{harm}} + z_{\text{obs}}, \qquad \chi(z) = \frac{\|z_{\text{obs}}\|}{\|z\|}.$$

Reviewer-facing note: This is the heart of the paper. Make it obvious that the method is trainable and not merely a reinterpretation of old logging code.

5. System and implementation

5.1 Implementation goal

Present BLT or a BLT-derived system as the instrumentation harness, not as the paper's sole contribution. The system should support:

- hook specification by block and hook name,
- activation capture,
- sparse-code extraction,
- restriction-map estimation or learning,
- interventions,
- cochain / Laplacian construction,
- export of traces and diagnostics.

5.2 Minimal implementation note

If the current codebase contains a random-projection or minimal sparse encoder, state explicitly that this is prototype infrastructure. The submission should either:

1. replace it with a learned encoder, or

2. clearly separate "prototype instrumentation" from "final learned method" in the experimental narrative.

5.3 Canonical configuration warning

[TODO: Normalize all model specifications before submission. Ensure layer count, hidden dimension, block count, and hook naming are consistent throughout the paper, code, tables, and appendix.]

Reviewer-facing note: A methods paper can survive a prototype implementation. It will not survive inconsistent model specs.

6. Experimental design

6.1 Baselines

At minimum compare against:

1. flat sparse transcoder / CLT baseline,
2. block-level transcoder with post hoc TDA sidecar,
3. identity-restriction sheaf baseline,
4. learned-restriction sheaf model.

6.2 Core experiments

A. Bridge necessity ablations

Intervene on bridge activations by zeroing, scaling, or clamping and measure change in prediction confidence, logit attribution, or task success. The question is whether bridge layers are structurally necessary for the task class under study.

B. Compression-forgetting curves

Track which tract-layer features survive into later blocks and which disappear under bounded-state compression. This directly tests the architectural distinction between compressed transport and exact retrieval.

C. Cross-block concept evolution

Build a block-level supernode graph in which each block contributes a set of active sparse features. Measure how a concept cluster transforms from tract regime to bridge regime and into the next block.

D. Obstruction localization

Compare sidecar Betti/coactivation summaries against exact sheaf-derived quantities such as stalk defect and nonzero H^1 generators. The claim is that the latter localize inconsistency more sharply.

E. Pure-attention versus hybrid comparison

Run the same pipeline on a pure-attention baseline. The expectation is that the sheaf structure becomes flatter and the tract/bridge distinction vanishes, reducing the explanatory advantage of block-local cohomological diagnostics.

6.3 Metrics

Metric	**Interpretation**
Reconstruction error	Fidelity of sparse local codes and decoder
Sparsity level	Degree of local chart compression
$\dim H^0$	Number of globally consistent section patterns
$\dim H^1$	Number of irreducible obstruction modes
Spectral gap	Robustness of global consistency
Stalk defect	Local contradiction at layer-token positions
Bridge necessity	Functional reliance on bridge layers
Obstruction energy χ	Fraction of representation outside harmonic/global component

Table 1: Primary quantitative metrics.

Reviewer-facing note: The experiments should test whether the formal objects explain something not already visible in flat sparse-feature baselines.

7. Results structure

Result question 1: Are the sheaf signals nontrivial?

Show that H^1, spectral gap, and stalk defect vary meaningfully across prompts, tasks, and interventions.

Result question 2: Do the signals align with architecture?

Show that tract and bridge layers occupy different roles, especially in block-local loops and bridge-mediated retrieval.

Result question 3: Do the signals improve interpretability?

Show sharper localization, clearer circuit decomposition, or better predictive differentiation than TDA-sidecar metrics.

Result question 4: Is the extra machinery justified?

Show either improved explanatory power at comparable cost or substantially better mechanistic diagnostics for modest additional overhead.

Figure 3 placeholder: Core quantitative result
Suggested panel: bridge ablation effect, obstruction energy shift, and spectral gap by task class.

Figure 3: Example core quantitative result figure.

8. Discussion

This section should place the method in the broader research hierarchy without letting that hierarchy swallow the paper.

A disciplined framing:

- **Geometric Data Science** is the umbrella.
- **Topological Data Science** is the engineering interface.
- **Sheaf transport and cohomology** are the formal core for the present method.

- **Classical TDA** remains useful as a downstream analysis layer, but is no longer the main representational object.

Reviewer-facing note: Keep the discussion short. The paper wins or loses on the method and experiments, not on the philosophical scope of the framing.

9. Limitations

1. **Prototype gap.** If the current system still uses approximate sidecars or minimal encoders, say so directly.
2. **Model-spec consistency.** The paper must standardize all architectural details before submission.
3. **Descriptive versus causal interpretation.** A nonzero obstruction class is evidence of inconsistency, not automatically evidence of a semantically meaningful causal mechanism.
4. **Scalability.** Exact sheaf operators may become expensive at large scale unless restriction maps and overlap structure are strongly structured.

10. Venue-specific positioning

10.1 TMLR

Lead with technical correctness, method clarity, and ablations. Emphasize that the paper introduces a principled representation-learning object and evaluates it carefully.

10.2 ICLR / NeurIPS

Lead with hybrid-language-model interpretability, architecture-aware representation learning, and benchmarkable experiments. Keep the introduction more empirical and less philosophical.

10.3 Topology or geometry companion paper

A separate companion paper can focus more heavily on:

- exact interpretation of sidecar metrics as sheaf approximations,
- holonomy over block-local loops,

- spectral theory of the sheaf Laplacian on hybrid covers,
- and the relation between persistence-style summaries and cohomological observables.

11. Figures and tables checklist

Essential figures

1. Hybrid 3:1 block cover and overlaps
2. Sheaf construction and cochain complex
3. Method pipeline
4. Core quantitative result
5. Cross-block concept evolution / supernode graph

Essential tables

1. Method component comparison
2. Quantitative metrics table
3. Intervention result summary

Method	**Sparse codes**	**Restriction maps**	**Exact cohomology**
Flat sparse transcoder	Yes	No	No
BLT + TDA sidecar	Yes	Implicit / none	No
Identity-sheaf baseline	Yes	Fixed identity	Partial
Learned-sheaf BLT	Yes	Learned	Yes

Table 2: Minimal comparison table to anchor the method space.

12. Reviewer objections and preemptive responses

"Why not just use a graph neural method?"

Because the core object is not merely adjacency but local data over overlaps with explicit transport between local representations. The restriction-map structure is the point.

"Is this just a reinterpretation of existing CLT machinery?"

No. The reinterpretation is only the first step. The actual method adds learned restriction maps, cohomological losses, Laplacian diagnostics, and block-local architectural priors.

"Why hybrids specifically?"

Because block-structured hybrid models induce meaningful overlap geometry and tract/bridge distinctions. In pure-attention models, that structure collapses toward a flatter regime.

"Where is the empirical value?"

The paper should answer with bridge ablations, compression-forgetting curves, cross-block concept evolution, and exact-vs-sidecar localization experiments.

13. Minimal conference-length page budget

For a 12-page conference version:

Section	Pages
Introduction	1.0
Background	1.0
Formal framework	2.0
Method	2.0
System	1.0
Experiments	2.0
Results	2.0
Discussion / Limitations	1.0

Closing sentence for the entire project

> We replace post hoc topological summaries of hybrid-model activations with a learned sheaf over block-local hook windows, turning consistency, obstruction, and bridge dependence into exact cohomological observables.

Next drafting move

Use this scaffold to produce the full conference manuscript in the following order:

1. lock the canonical model specification,
2. write the formal framework section cleanly,
3. define the learned restriction-map method and objective,
4. specify the ablation suite,
5. then write the introduction around the finished method rather than around the broad vision.